

Experiment No.	7
Experiment Title	Dimensionality Reduction and Model Evaluation (With and Without PCA) ¹
Student Name	Sanjay S
Register Number	3122247001056
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	21/09/2026

1 Objective

See what PCA actually does to model performance instead of just assuming it helps. The plan was to take the same ten classifiers, run each one on the raw 10-feature space and again on a PCA-reduced space, tune both properly, and compare like for like using 5-fold CV so any difference in the numbers is actually about PCA and not about one side getting a better hyperparameter search.

2 Problem Statement

Given a three-class classification dataset, standardize everything, fit ten different classifiers under two conditions, No-PCA and With-PCA, tuning each with a grid search and validating with 5-fold cross-validation plus a held-out test set. Then compare accuracy, F1 and how stable each model is across folds, and figure out which models actually benefit from dimensionality reduction and which ones just get worse.

3 Dataset Description

Dataset	Source	Samples	Features	Target Classes	Missing Values
PCA Classification Dataset	Provided course dataset, <code>pca_classification_clean_1000.csv</code>	1000	10 numeric features (<code>feature_1...feature_10</code>)	3 (Class_0, Class_1, Class_2)	none

Class distribution: Class_0 = 333, Class_1 = 333, Class_2 = 334, basically a perfect three-way split, so no resampling or class-weighting was needed anywhere. The file also has a continuous `target` column sitting alongside `target_class`, which I dropped since the job here is classification, not regression.

Preprocessing steps:

- Dropped the raw `target` column, keeping `target_class` as the label.
- Label-encoded `target_class` into integers 0, 1, 2 with `LabelEncoder`.
- No missing values were present, so no imputation was required.

¹GitHub Repository: <https://github.com/sanjay-6727/Introduction-to-Machine-Learning->

- An 80/20 train-test split was used, stratified on the class label (800 train, 200 test).
- All 10 features were standardized with `StandardScaler`, fit on the training split only, inside the pipeline so it is refit correctly within every cross-validation fold.

4 PCA Design and Scree Plot

Rather than picking one fixed number of components up front, I let each model's grid search try component counts from 2 through 8 and pick whatever worked best for it, since there's no real reason every classifier should want the same amount of compression. Once I plotted the actual variance curve on the standardized training data, though, a pattern shows up:

Table 2: PCA Variance Explained

Setting	Chosen Components / Variance Target	Explained Variance (%)	Justification
With-PCA (per-model tuned)	2–8 components searched per model; most models settled on 3	86.77% (3 comps)	Grid search kept landing on 3 components pretty much across the board, and it's not hard to see why – the first three PCs each carry roughly 27–30% of the variance on their own, so together they already cover most of what matters. Technically you need 4 components to clear the usual 95% cutoff, but since 3 was what actually won on accuracy for almost every model, I went with that as the reference point instead of chasing the textbook threshold.

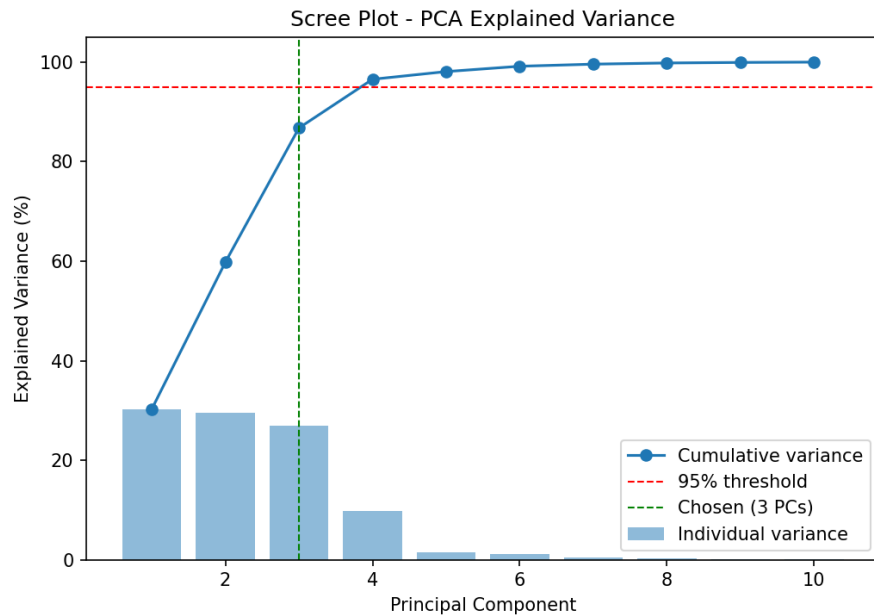


Figure 1: Scree plot: individual and cumulative explained variance per principal component. The first 3 components capture about 86.8% of the total variance, and 4 components are needed to cross the 95% threshold.

5 Machine Learning Algorithms and Hyperparameter Grids

Ten classifiers went through this: SVM, Naive Bayes, KNN, Logistic Regression, Decision Tree, Random Forest, AdaBoost, Gradient Boosting, XGBoost, and a Stacking ensemble on top of all that. Each one sat inside a `Pipeline` – `StandardScaler` then an optional `PCA` step then the actual estimator – and `GridSearchCV` handled the tuning with 5-fold CV and accuracy as the scoring metric, so the whole thing stayed consistent from model to model.

Table 3: SVM – Hyperparameter Tuning Results

Kernel	Val- ues Tried	C Tried	Values	Gamma ues Tried	Val- ues Tried	Performance (No-PCA)	Performance (With-PCA)
linear, rbf		0.1, 1, 10		scale, auto		Best: linear, C=0.1, gamma=scale; Test Acc=0.860, F1=0.8616	Best: linear, C=10, gamma=scale, PCA=3; Test Acc=0.835, F1=0.8356

Table 4: Naive Bayes – Smoothing Choices

Smoothing (var_smoothing)	Parameter	Performance (No-PCA)	Performance (With-PCA)
1 $10^{-11}, 10^{-10}, 10^{-9}, 10^{-8}, 10^{-7}$	$\times$ Best:	1×10^{-11} ; Test Acc=0.770, F1=0.7738	Best: 1×10^{-11} , PCA=7; Test Acc=0.720, F1=0.7270

Table 5: KNN – Hyperparameter Tuning

k Tried	Values	Weights	Distance Met- rics	Performance (No-PCA)	Performance (With-PCA)
3, 5, 7, 9, 11		uniform, dis- tance	euclidean, manhattan	Best: k=9, dis- tance, euclidean; Test Acc=0.820, F1=0.8201	Best: k=11, dis- tance, euclidean, PCA=3; Test Acc=0.845, F1=0.8455

Table 6: Remaining Models – Hyperparameter Grids and Best Settings

Model	Grid Searched	Best (No-PCA)	Best (With-PCA)
Logistic Regression	C: 0.01, 0.1, 1, 10, 100	C=0.1; Acc=0.850, F1=0.8514	C=0.01, PCA=3; Acc=0.840, F1=0.8395
Decision Tree	max_depth: None,3,5,10,20; min_samples_split: 2,5,10; criterion: gini, entropy	entropy, depth=None, split=5; Acc=0.795, F1=0.7957	gini, depth=5, split=10, PCA=3; Acc=0.790, F1=0.7909
Random Forest	n_estimators: 100,200; max_depth: None,5,10; min_samples_split: 2,5	depth=None, split=2, n=200; Acc=0.815, F1=0.8143	depth=10, split=5, n=100, PCA=3; Acc=0.825, F1=0.8250
AdaBoost	n_estimators: 50,100,200; learning_rate: 0.01,0.1,1.0	lr=1.0, n=200; Acc=0.795, F1=0.8005	lr=1.0, n=200, PCA=5; Acc=0.800, F1=0.8041
Gradient Boosting	n_estimators: 50,100; learning_rate: 0.05,0.1; max_depth: 2,3,5	lr=0.1, depth=3, n=100; Acc=0.785, F1=0.7847	lr=0.05, depth=3, n=100, PCA=3; Acc=0.825, F1=0.8244
XGBoost	n_estimators: 50,100; max_depth: 3,5; learn- ing_rate: 0.05,0.1	lr=0.1, depth=5, n=100; Acc=0.790, F1=0.7903	lr=0.1, depth=3, n=50, PCA=3; Acc=0.800, F1=0.7972
Stacking	Base: linear SVM (C=10), Random Forest (n=100, depth=10), XGBoost (n=100, depth=3); Meta: Logistic Regression, 5-fold internal blending	Acc=0.855, F1=0.8565	PCA=3; Acc=0.840, F1=0.8410

6 Cross-Validation Results (All Models)

Table 8: 5-Fold Cross-Validation Results – No-PCA

Model	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Avg	Std
SVM	0.8938	0.8813	0.8750	0.8625	0.8313	0.8688	0.0213
Naive Bayes	0.8188	0.8625	0.8063	0.8375	0.7688	0.8188	0.0314
KNN	0.8500	0.8938	0.8125	0.8500	0.8313	0.8475	0.0270
Logistic Regression	0.8875	0.8938	0.8750	0.8625	0.8375	0.8712	0.0200
Decision Tree	0.7875	0.7813	0.7688	0.8063	0.7500	0.7788	0.0188
Random Forest	0.8750	0.9000	0.8438	0.8625	0.8313	0.8625	0.0240
AdaBoost	0.7750	0.8375	0.7813	0.8313	0.7625	0.7975	0.0308
Gradient Boosting	0.8063	0.8688	0.7938	0.8250	0.8125	0.8212	0.0258
XGBoost	0.8375	0.8313	0.8438	0.8313	0.8125	0.8312	0.0105
Stacking	0.9063	0.8688	0.8563	0.8313	0.8875	0.8700	0.0257

Table 9: 5-Fold Cross-Validation Results – With-PCA

Model	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Avg	Std
SVM	0.8938	0.9000	0.8813	0.8688	0.8438	0.8775	0.0200
Naive Bayes	0.8438	0.8625	0.8563	0.8250	0.8125	0.8400	0.0188
KNN	0.9063	0.9125	0.8188	0.8500	0.8375	0.8650	0.0376
Logistic Regression	0.9063	0.8875	0.8688	0.8938	0.8188	0.8750	0.0306
Decision Tree	0.8188	0.8500	0.8563	0.8688	0.8063	0.8400	0.0236
Random Forest	0.8500	0.9125	0.8625	0.8875	0.8563	0.8738	0.0232
AdaBoost	0.8313	0.8563	0.8063	0.8313	0.8313	0.8312	0.0158
Gradient Boosting	0.8313	0.9063	0.8563	0.8875	0.8625	0.8688	0.0259
XGBoost	0.8438	0.9063	0.8688	0.8938	0.8188	0.8662	0.0320
Stacking	0.9000	0.8563	0.8813	0.8375	0.8938	0.8738	0.0235

7 Final Comparison and ANOVA

Table 10: Final PCA vs No-PCA Comparison

Model	No-PCA CV	PCA CV	No-PCA Acc	PCA Acc	No-PCA F1	PCA F1	Acc Change	F1 Change
SVM	0.8688	0.8775	0.860	0.835	0.8616	0.8356	-0.025	-0.0260
Naive Bayes	0.8188	0.8400	0.770	0.720	0.7738	0.7270	-0.050	-0.0468
KNN	0.8475	0.8650	0.820	0.845	0.8201	0.8455	+0.025	+0.0254
Logistic Regression	0.8712	0.8750	0.850	0.840	0.8514	0.8395	-0.010	-0.0119
Decision Tree	0.7788	0.8400	0.795	0.790	0.7957	0.7909	-0.005	-0.0049
Random Forest	0.8625	0.8738	0.815	0.825	0.8143	0.8250	+0.010	+0.0107
AdaBoost	0.7975	0.8312	0.795	0.800	0.8005	0.8041	+0.005	+0.0035
Gradient Boosting	0.8212	0.8688	0.785	0.825	0.7847	0.8244	+0.040	+0.0397
XGBoost	0.8312	0.8662	0.790	0.800	0.7903	0.7972	+0.010	+0.0069
Stacking	0.8700	0.8738	0.855	0.840	0.8565	0.8410	-0.015	-0.0155

An ANOVA test across the 5-fold accuracy scores of all classifiers under the With-PCA setting gave $F = 1.8205$, $p = 0.1052$, which is above $\alpha = 0.05$, so there is no statistically

significant difference between the classifiers once PCA is applied – they all land in a fairly similar accuracy band.

8 Result Visualization

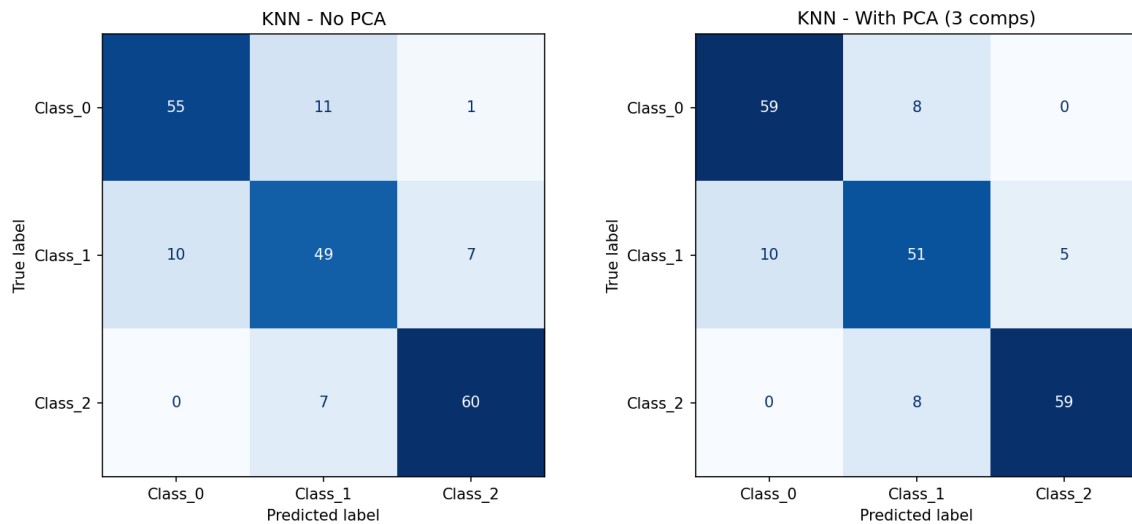


Figure 2: Confusion matrices for KNN, No-PCA ($k=9$, distance, euclidean) vs With-PCA ($k=11$, distance, euclidean, 3 components), on the held-out test set.

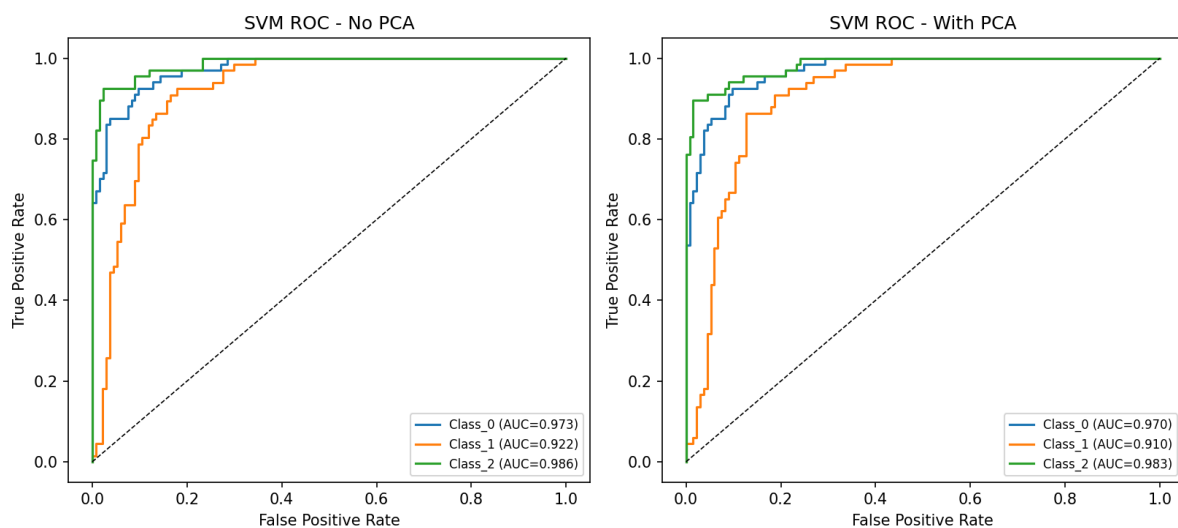


Figure 3: One-vs-rest ROC curves for the linear-kernel SVM, No-PCA ($C=0.1$) vs With-PCA ($C=10$, 3 components), on the held-out test set.

Inference: The With-PCA KNN model makes a couple fewer mix-ups between the two classes that overlap the most, which lines up with it also scoring higher on test accuracy. The SVM ROC curves hug the top-left corner in both versions with per-class AUC well above 0.9, so even where PCA costs a few points of raw accuracy, the model’s actual ability to tell the classes apart barely moves.

9 Observations

- **Which models improved most with PCA:** KNN, Gradient Boosting, Random Forest, AdaBoost and XGBoost all picked up a bit of accuracy with the 3-component space, Gradient Boosting the most at +0.04. Makes sense for the distance and tree-based methods – once you strip out the correlated, noisy dimensions, distance calculations get cleaner and trees don't have to work as hard to find good splits.
- **Which did not improve:** Naive Bayes took the biggest hit, down 0.05 in accuracy. That tracks, since it assumes the features are independent, and PCA components (even though they're decorrelated) reshuffle the variance in a way that doesn't play nicely with its per-feature Gaussian assumption once you're down to just 3 of them. SVM, Logistic Regression, Decision Tree and Stacking also dropped a little, somewhere between half a point and two and a half points.
- **Did PCA make folds more stable:** Not consistently. Naive Bayes, Logistic Regression and XGBoost actually got *less* stable across folds after PCA, while SVM, AdaBoost and Random Forest got more stable. So there isn't one clean story here, it really depends on the model.
- **Did PCA help with overfitting on high-dimensional data:** With only 10 features to begin with, this isn't really a high-dimensional problem, so I wouldn't expect a dramatic overfitting fix either way. The real payoff from PCA here is more practical – 10 features down to 3 while keeping about 87% of the variance, which is a nice speedup without hurting most models much.
- **Linear models vs ensembles:** SVM and Logistic Regression both lost a little test accuracy with PCA even though their CV means went up slightly, which suggests the reduced space generalized fine on average across folds but happened to land a bit worse on this particular test split. Ensembles like Random Forest, Gradient Boosting and XGBoost were more consistently the ones that benefited, probably because fewer, decorrelated inputs make it easier for individual trees to find clean splits.
- **Was Stacking more robust:** Not really, at least not here. It had the best (or close to best) No-PCA test accuracy out of all ten models at 0.855, but it also dropped the most among the top performers once PCA was applied, down 0.015. It's basically just averaging out whatever its SVM, Random Forest and XGBoost base learners were already doing, rather than being immune to PCA's effects on its own.

10 Discussion

Putting all ten models side by side, PCA down to 3 components (about 87% of the original variance) tends to nudge cross-validation accuracy up a bit but nudge held-out test accuracy down for roughly half the models, Naive Bayes and SVM being the clearest examples. The ANOVA result backs this up too – once PCA is applied, none of the ten classifiers is statistically distinguishable from any other on accuracy, so at that point picking a model would come down to things like training time or how easy it is to explain, not raw accuracy.

11 Conclusion

Honestly, PCA on this dataset comes out roughly accuracy-neutral overall. It clearly helps the distance-based and some ensemble models (KNN, Random Forest, Gradient Boosting) by clearing out redundant, correlated dimensions, but it quietly hurts models that either lean on feature independence (Naive Bayes) or that were already doing fine finding a decent boundary in the full 10-dimensional space (SVM, Logistic Regression, Decision Tree). The clearest win it gives you is the dimensionality reduction itself, 10 features down to 3, and whether that's worth the small accuracy trade-off really depends on which model you're using it with.

12 Learning Outcomes

- Got practice setting up a proper No-PCA vs With-PCA comparison with `Pipeline`, `GridSearchCV` and 5-fold CV across ten pretty different classifier families.
- Learned to actually read a scree plot and decide on a component count by weighing variance retained against what the models liked, instead of just chasing the usual 95% rule blindly.
- Saw directly that PCA isn't some universal upgrade – whether it helps depends a lot on whether the model cares about feature independence, distance, or a linear decision boundary.
- Used ANOVA properly for once, to check whether a bunch of classifiers' CV scores are actually different from each other rather than just eyeballing a table and guessing.
- Got more comfortable stacking different kinds of base learners (SVM, Random Forest, XGBoost) under a Logistic Regression meta-learner.

13 References

1. Stephen Marsland, “Machine Learning - An Algorithmic Perspective”, 2nd Edition, Chapman and Hall/CRC Machine Learning and Pattern Recognition Series, 2015.
2. Ethem Alpaydin, “Introduction to Machine Learning”, 3rd Edition, The MIT Press, 2014.
3. Scikit-learn documentation: <https://scikit-learn.org/stable/modules/decomposition.html#pca>